



ALGORYTMY I STRUKTURY DANYCH

Termin lab.: piątek 13:30

Kierunek: Informatyka

Grupa lab.: 5

SPRAWOZDANIE Z LABORATORIUM

1. Algorytmy Sortowania

Autorzy:

PRO100KatYT

Burdziq

Prowadzący zajęcia:

mgr inż. Domomod

Rok akademicki 2025/2026

Spis treści

1	Wprowadzenie i krótkie opisy algorytmów	2
1.1	Sortowanie przez wstawianie (Insertion Sort)	2
1.2	Sortowanie bąbelkowe (Bubble Sort)	2
1.3	Sortowanie przez wybór (Selection Sort)	2
1.4	Sortowanie Shella (Shell Sort)	2
1.5	Sortowanie przez zliczanie (Counting Sort)	2
1.6	Sortowanie przez scalanie (Merge Sort)	2
1.7	Sortowanie szybkie (Quick Sort)	2
1.8	Sortowanie przez kopcowanie (Heap Sort)	2
2	Złożoność czasowa - porównanie między algorytmami	3
2.1	Zestawienie pomiarów (zbiór danych losowych)	3
2.2	Zbiorcze wykresy dla danych losowych	3
3	Złożoność czasowa - optymistyczna i pesymistyczna z wykresami	4
3.1	Insertion Sort	4
3.2	Bubble Sort	4
3.3	Selection Sort	5
3.4	Shell Sort	5
3.5	Counting Sort	5
3.6	Merge Sort	6
3.7	Quick Sort (pivot skrajnie lewy)	6
3.8	Quick Sort (pivot losowy)	6
3.9	Heap Sort	7
4	Podsumowanie	7

1 Wprowadzenie i krótkie opisy algorytmów

W ramach niniejszego sprawozdania zaimplementowaliśmy w języku Python osiem algorytmów sortowania (zamiast wymaganych „tylko” sześciu :)) i je przetestowaliśmy. Są to kolejno:

1.1 Sortowanie przez wstawianie (Insertion Sort)

Algorytm dzieli tablicę na dwie części: posortowaną (początkowo zawierającą tylko pierwszy element) oraz nieposortowaną. I w każdej iteracji pobiera pierwszy element z części nieposortowanej i przesuwając go w lewo tak długo, aż znajdzie dla niego odpowiednie miejsce w części posortowanej.

1.2 Sortowanie bąbelkowe (Bubble Sort)

W naszym mniemaniu najłatwiejszy algorytm do implementacji. Polega na przechodzeniu przez tablicę i porównywaniu sąsiednich elementów. Jeśli są one w złej kolejności, to zamienia je. Każda iteracja kończy się „wypłynięciem” na koniec tablicy największego elementu niczym bąbelek powietrza.

1.3 Sortowanie przez wybór (Selection Sort)

Algorytm co iterację przeszukuje nieposortowaną część tablicy i znajduje najmniejszy element. Następnie zamienia go miejscami z pierwszym elementem tej nieposortowanej części. I tak aż do pełnego posortowania ciągu.

1.4 Sortowanie Shella (Shell Sort)

Jest to lepsza wersja sortowania przez wstawianie. Zamiast porównywać elementy sąsiadujące, algorytm porównuje elementy oddalone od siebie o przyrost stopniowo zmniejszany aż do jedynki, czyli klasycznego sortowania przez wstawianie. Użyliśmy zgodnie z poleceniem przyrostów Sedgewicka, korzystając ze wzoru $4^{k+1} + 3 \cdot 2^k + 1$.

1.5 Sortowanie przez zliczanie (Counting Sort)

Ten algorytm zlicza wystąpienia poszczególnych wartości w dodatkowej tablicy pomocniczej. Następnie modyfikuje tę tablicę, obliczając tzw. „sumy prefiksowe” co pozwala precyzyjnie ustalić indeksy poszczególnych elementów w tablicy końcowej. Bez tego ten algorytm nie byłby stabilny.

1.6 Sortowanie przez scalanie (Merge Sort)

Algorytm wykorzystujący słynne „dziel i zwyciężaj”. Dzieli tablicę na coraz mniejsze połówki aż nie będą jednoelementowe. Następnie iteracyjnie scala je ze sobą w odpowiedniej kolejności, tworząc coraz większe posortowane bloki.

1.7 Sortowanie szybkie (Quick Sort)

Jeden z pewnie najbardziej rozpoznawalnych algorytmów opartych też na podziale. Przesłania elementy tablicy tak, by wartości mniejsze od wybranego pivota znalazły się po jego lewej stronie, a większe po prawej. Następnie rekurencyjnie sortuje obie części powtarzając to co napisaliśmy.

1.8 Sortowanie przez kopcowanie (Heap Sort)

Przódę przerabia wejściową tablicę w kopiec max-heap, gdzie każdy rodzic jest większy od swoich dzieci. Następnie iteracyjnie zdejmuję największy element z samego szczytu kopca, przenosi go na koniec tablicy i naprawia strukturę tego drzewa.

2 Złożoność czasowa - porównanie między algorytmami

2.1 Zestawienie pomiarów (zbiór danych losowych)

Poniższa tabela prezentuje czas sortowania (w sekundach) w zależności od rozmiaru tablicy n . Dla lepszej czytelności użyliśmy w niej angielskich nazw algorytmów i wybranych wartości n .

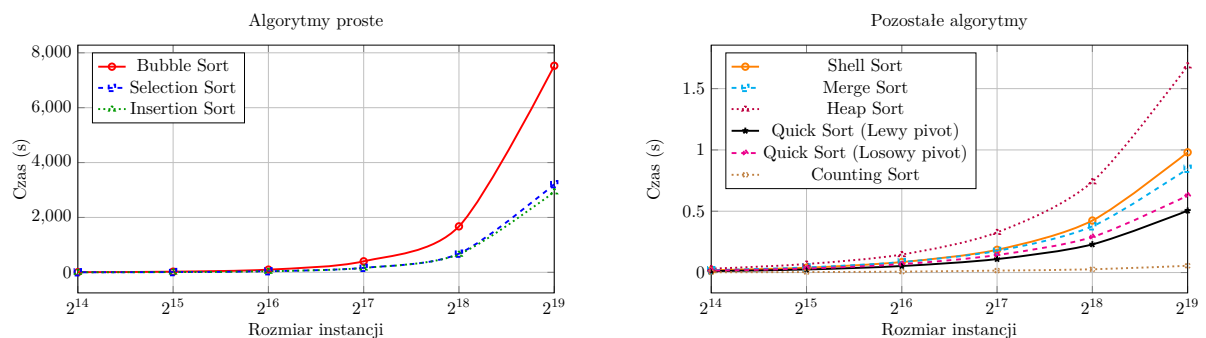
Algorytm / Rozmiar (n)	2^9	2^{12}	2^{14}	2^{16}	2^{18}	2^{19}
Bubble Sort	0,0043	0,3645	6,0885	98,3080	1674,7388	7527,2910
Selection Sort	0,0021	0,1464	2,3930	39,5991	680,9892	3211,4377
Insertion Sort	0,0018	0,1446	2,3365	39,2116	660,2191	2932,7831
Heap Sort	0,0005	0,0057	0,0318	0,1469	0,7406	1,6863
Shell Sort	0,0003	0,0033	0,0188	0,0854	0,4242	0,9806
Merge Sort	0,0004	0,0037	0,0193	0,0855	0,3770	0,8463
Quick Sort (losowy pivot)	0,0004	0,0033	0,0158	0,0682	0,2895	0,6276
Quick Sort (lewy pivot)	0,0002	0,0026	0,0116	0,0532	0,2296	0,5043
Counting Sort	0,0004	0,0007	0,0023	0,0074	0,0263	0,0551

Tabela 1: Czasy sortowania dla ciągów losowych wybranej wielkości.

Jak można zauważyć, dla najwolniejszych algorytmów testy dla największych tablic trwały bardzo długo (nawet ponad 2 godziny dla Bubble Sorta). Warto dodać jako ciekawostkę, że wszystkie testy wszystkich algorytmów na wszystkich zbiorach danych (od 2^2 -elementowych do 2^{19} -elementowych) łącznie zajęły około 20 godzin mniej więcej od godziny 16:00 do 12:00 dnia następnego.

2.2 Zbiorcze wykresy dla danych losowych

W poniższym zestawieniu rozbiliśmy wyniki na dwa osobne wykresy: jeden dla algorytmów prostych, a drugi dla reszty algorytmów. Zlogarytmowaliśmy oś X i rozpoczęliśmy ją od 2^{14} , celem zwiększenia czytelności wykresu.

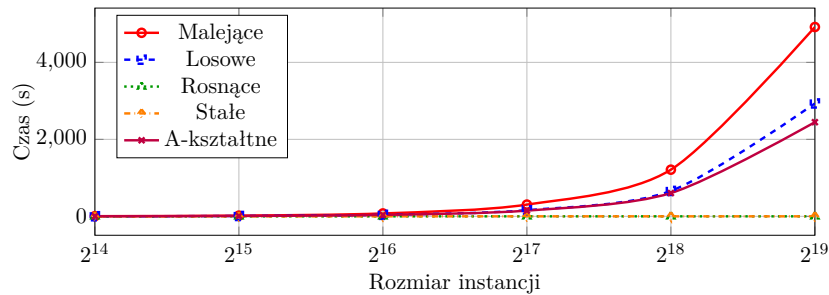


Rysunek 1: Wykresy porównawcze czasów sortowania danych losowych.

3 Złożoność czasowa - optymistyczna i pesymistyczna z wykresami

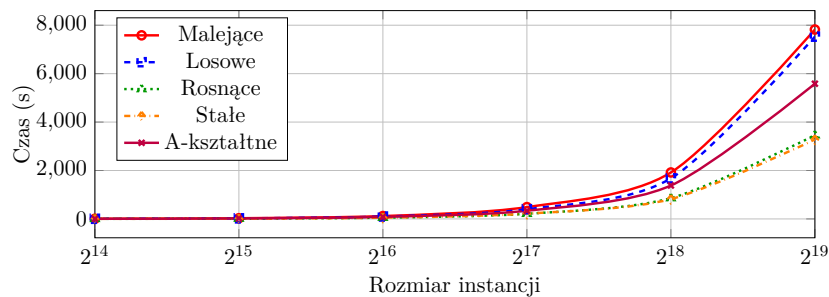
Ten rozdział poświęcony jest analizie wpływu ułożenia ciągów danych (losowo, rosnąco, malejąco, A-kształtne oraz o stałych wartościach) na wydajność algorytmów. Dzięki temu można wyznaczyć złożoności optymistyczne i pesymistyczne.

3.1 Insertion Sort



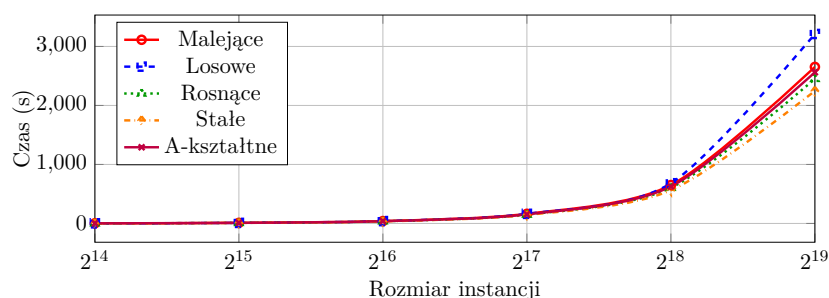
Analiza: Dla danych malejących (czyli najgorszego przypadku) i losowych czas rośnie bardzo szybko, pokazując złożoność $\mathcal{O}(n^2)$. Z kolei dla danych rosnących oraz o stałych wartościach algorytm jest znacznie szybszy, ponieważ wewnętrzna pętla natychmiast się kończy co redukuje złożoność do liniowej $\mathcal{O}(n)$.

3.2 Bubble Sort



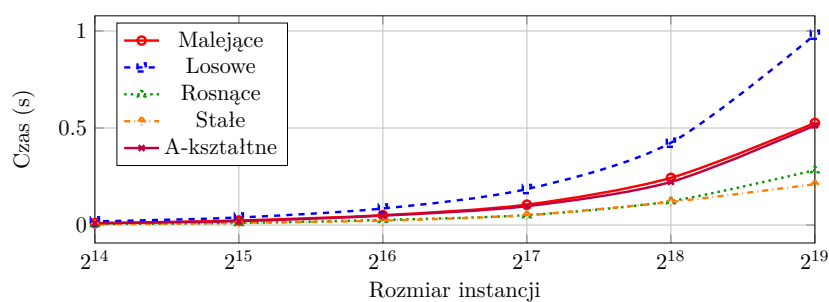
Analiza: Przy dużych rozmiarach tablic dla danych malejących oraz losowych czas rośnie bardzo szybko, zgodnie ze złożonością $\mathcal{O}(n^2)$. Dla danych rosnących oraz stałych czas rośnie wolniej, jednak nadal złożoność wynosi $\mathcal{O}(n^2)$. Sortowanie bąbelkowe jest zdecydowanie najbardziej czasochłonnym rodzajem sortowania.

3.3 Selection Sort



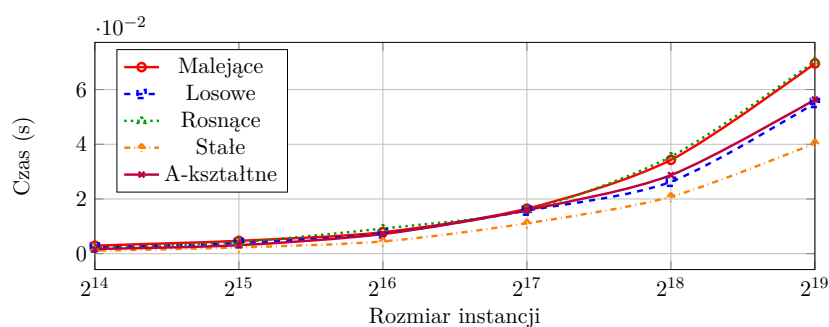
Analiza: Dla danych losowych algorytm wyróżnia się największym czasem sortowania. Trochę mniej czasu zajmuje posortowanie tablic zawierających dane malejące, rosnące czy stałe. Dla każdego przypadku złożoność obliczeniowa algorytmu wynosi $\mathcal{O}(n^2)$.

3.4 Shell Sort



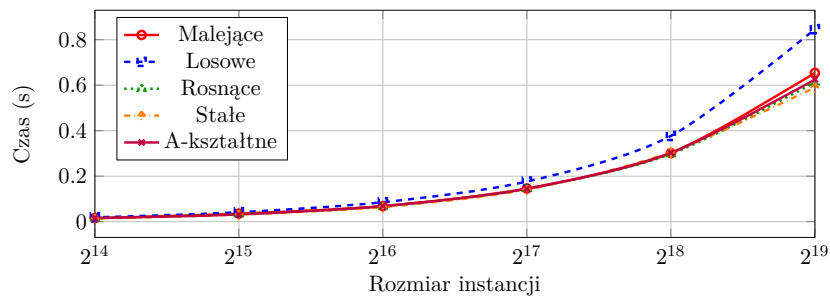
Analiza: Ten rodzaj sortowania jest zdecydowanie szybszy od wcześniej omówionych sortowań. Najwięcej czasu zajmuje mu posortowanie danych występujących w kolejności losowej. Trochę mniej czasu zajmuje posortowanie danych malejących. Dane rosnące oraz stałe zajmują algorytmowi najmniej czasu. Pesymistyczna złożoność obliczeniowa całego algorytmu wynosi $\mathcal{O}(n^2)$, optymistyczna $\mathcal{O}(n \log n)$, a średnia $\mathcal{O}(n^{1.25})$.

3.5 Counting Sort



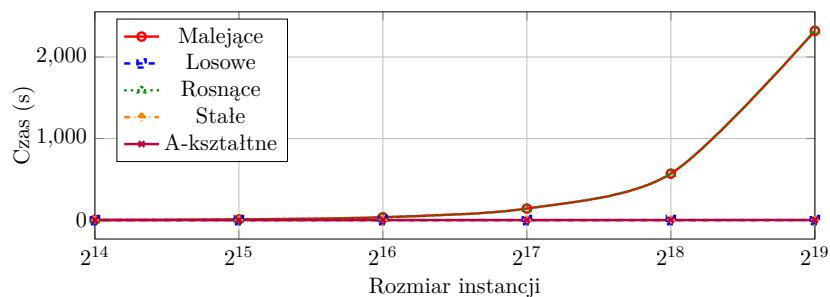
Analiza: Jest to najszybszy rodzaj sortowania spośród wszystkich omawianych w tym sprawozdaniu. Najwięcej czasu zajmuje mu posortowanie danych malejących oraz rosnących, a najmniej danych stałych. Złożoność obliczeniowa tego algorytmu wynosi $\mathcal{O}(n)$.

3.6 Merge Sort



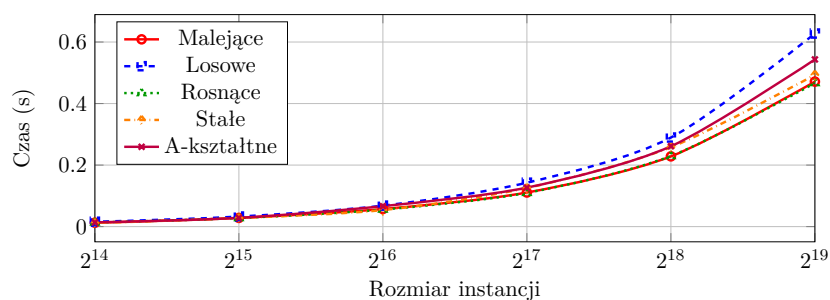
Analiza: Najwięcej czasu zajmuje posortowanie danych losowych, a niewiele mniej danych malejących, rosnących i stałych. Złożoność obliczeniowa tego algorytmu wynosi $\mathcal{O}(n \log n)$.

3.7 Quick Sort (pivot skrajnie lewy)



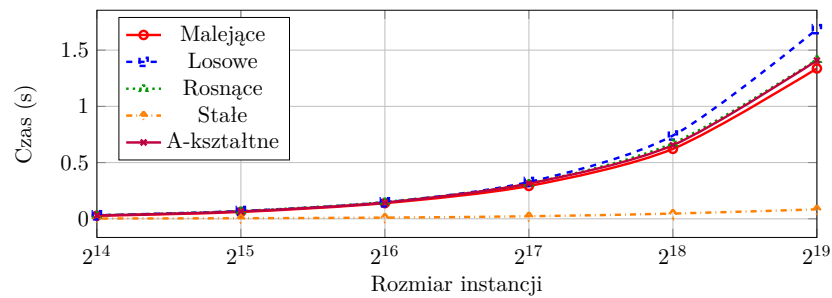
Analiza: Dla danych malejących, rosnących oraz stałych algorytm ten jest najwolniejszy i wyróżnia się na tle pozostałych rodzajów danych. Przyjmuje złożoność obliczeniową $\mathcal{O}(n^2)$. Dla danych losowych złożoność obliczeniowa wynosi $\mathcal{O}(n \log n)$.

3.8 Quick Sort (pivot losowy)



Analiza: Ten rodzaj Quick Sorta jest zdecydowanie mniej czasochłonny od Quick Sorta z pivotem lewym. Różnice między poszczególnymi danymi są w tym przypadku niewielkie. Algorytm jest jednak najszybszy dla danych malejących i rosnących, troszkę wolniejszy dla stałych, a najwolniejszy z kolei dla losowych. Złożoność obliczeniowa tego algorytmu wynosi $\mathcal{O}(n \log n)$.

3.9 Heap Sort



Analiza: Ten rodzaj sortowania jest najwolniejszy dla danych losowych. Nieco szybszy jest dla danych rosnących oraz malejących. Zdecydowanie najszybszy jest dla danych stałych. Jego złożoność obliczeniowa wynosi $\mathcal{O}(n \log n)$.

4 Podsumowanie

W sprawozdaniu przedstawiliśmy 8 algorytmów sortujących. Porównaliśmy je pod względem czasu sortowania i złożoności obliczeniowej. Najszybszym okazał się być Counting Sort, a najwolniejszym Bubble Sort. Rozstrzygnęliśmy także, że rozkład danych wejściowych wpływa na działanie algorytmu. Dla części algorytmów istnieją przypadki pesymistyczne, optymistyczne i średnie w zależności od rodzaju danych. Wykonanie tych zadań pozwoliło nam uporządkować oraz usystematyzować wiedzę na temat algorytmów sortowania i przećwiczyć umiejętności programistyczne.